



Eppy-LLM: An Agentic Workflow for Language-Driven Building Energy Modeling and Optimization Using EnergyPlus

Huiwen Zhou¹, Liang Zhang^{1,2},

¹University of Arizona, Tucson, AZ, USA

²National Laboratory of the Rockies, Golden, CO, USA

Abstract

This study introduces Eppy-LLM, a lightweight multi-agent framework that integrates large language models (LLMs) with EnergyPlus to support interpretable, reproducible, and adaptive analysis based on building energy modeling. The system translates natural-language design objectives into deterministic simulation workflows through a structured sequence of three agents: a semantic interpreter, a rule-based orchestrator, and an analytical feedback module. Validated using NREL's iUnit building model, the framework achieved 100% syntax-valid simulations and demonstrated adaptive parameter selection across diverse goals, including cooling load reduction, daylight optimization, and total energy minimization. These results underscore the feasibility of bridging natural-language reasoning with physics-based simulation, paving the way for human-centered and explainable design automation.

Introduction

1. Introduction

1.1 Background and Motivation

Buildings account for nearly 29% of total end-use energy consumption in the United States, rising to approximately 40% when generation losses are included (U.S. EIA 2022). Improving building energy performance is therefore critical to achieving decarbonization targets. During the conceptual design phase, early architectural decisions largely determine energy outcomes, yet performance evaluation at this stage remains limited.

Building Energy Modeling (BEM) tools like EnergyPlus and OpenStudio offer detailed control over thermal, HVAC, and envelope parameters, enabling high-fidelity simulation of building performance. Despite the maturity of simulation engines such as

EnergyPlus, BEM workflows remain labor-intensive. Modelers frequently conduct repetitive IDF edits, parameter sweeps, and sensitivity studies, and must manually document assumptions and results to maintain auditability. These processes impose substantial time and coding overhead, limiting rapid exploration of design alternatives, particularly when multiple design objectives must be evaluated under tight design timelines in early decision making stage.

Efforts to lower this barrier have included rule-based automation, parametric modeling, and surrogate machine learning approaches (Zhang et al. 2021). While these methods simplify repetitive tasks, they remain largely static and script-driven, requiring explicit coding or predefined templates. They lack semantic understanding of design intent and cannot flexibly adapt to evolving goals. As a result, energy modeling continues to be expert-centric rather than interactive and accessible.

1.2 Emergence of AI and Large Language Models in Engineering and Building Design

Recent advances in artificial intelligence particularly LLMs, offer new opportunities to bridge unstructured design intent with structured engineering models (Vaithilingam et al. 2022; Zhang et al. 2023; Wang & Chen 2023). LLMs demonstrate strong capabilities in semantic understanding, contextual reasoning, and code generation, making them powerful tools for human-computer collaboration in technical domains (Das et al. 2024). Unlike traditional rule-based systems, LLMs can interpret natural-language objectives, decompose them into executable steps, and generate structured outputs suitable for simulation or optimization.

In the Architecture, Engineering, and Construction (AEC) domain, emerging research has shown the feasibility of applying LLMs for modeling and analysis automation. Examples include automatic generation and debugging of EnergyPlus input files from textual descriptions (Zhang et al. 2025), translation of 2D

drawings into 3D BIM models (Zhou et al. 2024), and automation of CFD simulations using OpenFOAM (Pandey et al. 2025). LLM-based agentic frameworks have also been explored for smart-building control (Yan et al. 2025; Yang et al. 2024; Rivkin et al. 2024; Kalyuzhnaya et al. 2025), semantic data modeling (e.g., BrickLLM), and code-compliance checking (e.g., LLM-QueryBC) (Zhu et al. 2024; Perini et al. 2025). These developments signal a paradigm shift—from static text-to-model generation toward semantic reasoning with interactive feedback. They suggest the potential for intelligent design assistants that interpret qualitative goals and autonomously modify BEM parameters to achieve energy-efficient outcomes (Das et al. 2024).

1.3 Research Gap and Objectives

Despite these promising developments, most existing LLM-based frameworks for building energy applications remain one-directional focused on initial model generation without iterative reasoning or closed-loop feedback (Zhang et al. 2025; Khadka 2025). Validation typically relies on static case examples rather than adaptive workflows. As a result, architects without engineering expertise lack mechanisms to translate semantic design objectives into quantitative parameter adjustments and observe corresponding performance outcomes in real time. This absence of accessible feedback loops limits both usability and reliability in practical design scenarios (Yu et al. 2025; Zhang et al. 2025).

To address these limitations, this study proposes Eppy-LLM, an interactive, language-driven framework that integrates LLM reasoning with EnergyPlus simulation through structured tool interfaces. The framework establishes a closed-loop connection between natural-language intent, parametric model modification, and simulation-based feedback, enabling human-in-the-loop optimization for early-stage building design. By combining semantic interpretation, rule-based simulation management, and quantitative analysis agents, Eppy-LLM aims to improve productivity, repeatability, and auditability in EnergyPlus workflows, enabling faster goal-driven exploration while keeping domain experts in the loop.

2. Methodology

In BEM, natural-language interfaces offer convenience but introduce non-deterministic behaviors that can reduce trust and auditability. A single LLM agent that directly edits simulation models may produce inconsistent or invalid IDF modifications. To address this, we adopt a three-agent architecture that separates

intent interpretation, constrained model editing, and performance analysis. Specifically, Agent 1 translates linguistic goals into a structured set of candidate parameters; Agent 2 applies deterministic, tool-bound modifications limited to pre-defined IDF fields; and Agent 3 executes EnergyPlus simulations and provides quantitative feedback. This modular design improves reproducibility, reduces hallucination risk, and enables traceable end-to-end workflows suitable for professional energy modelers.

LLM-based reasoning is inherently stochastic and may generate variable suggestions across runs. In contrast, building performance simulation workflows require deterministic and verifiable model changes to support professional decision-making. Therefore, we design the workflow such that the LLM is primarily used for intent interpretation and parameter recommendation, while all IDF edits and simulation executions are performed deterministically via tool-bound procedures and validation checks.

2.1 System Overview

Eppy-LLM is a lightweight agent-based optimization framework designed to assist architects during early-stage envelope design. It enables performance-driven design exploration without requiring prior expertise in building physics and building energy simulation. By accepting a baseline EnergyPlus input data file (IDF) and a natural-language design goal, the framework orchestrates a multi-agent workflow that interprets user intent, modifies model parameters, performs simulations, and synthesizes interpretable feedback. Eppy is an open-source Python library that enables programmatic reading, editing, and writing of EnergyPlus IDF files (Philip, 2026).

The architecture comprises three modular agents: semantic interpretation (Agent 1), rule-based orchestration (Agent 2), and analytical feedback (Agent 3). Agent 1 converts user objectives into structured parameter schemas by semantically parsing the IDF. Agent 2 translates these schemas into simulation-ready model variants using deterministic modification rules. Agent 3 executes EnergyPlus simulations and analyzes the results to extract both qualitative and quantitative insights, closing the human-machine-simulation loop.

Each agent operates through a defined computational interface, exchanging data via standardized JSON files and organized directory hierarchies. This design ensures transparency, reproducibility, and modular execution. The framework aims to make high-level design feedback accessible through natural-language interaction, enabling iterative refinement of envelope configurations based on performance evidence.

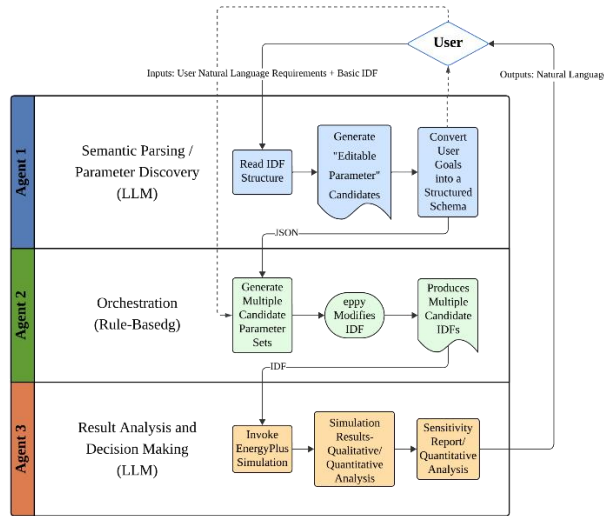


Figure 1. Agentic Workflow for BEM Automation

2.2 Agent 1: Semantic Interpreter

Agent 1 serves as a semantic interpreter that connects user-defined natural-language design goals with simulation-ready building model parameters. As illustrated in the Semantic Parsing stage of Figure 1, Agent 1 translates qualitative user objectives into a structured parameter schema by dynamically parsing the EnergyPlus Input Data File (IDF) and querying a local LLM. List 2.1 shows the abbreviated standard prompt response format used by Agent 1, including both the editable object samples provided to the LLM and the required JSON output schema.

List 2.1. Abbreviated LLM Prompt Specification for Agent 1 (Full implementation available in project repository.)

```
Prompt = f""" System role: "You are a
Building Energy Modeling expert."
Input variables:
  {user_goal} = user's natural-language
design intent.
  {editable_json} = IDF editability map
organized by categories (e.g., Roof, Wall,
Window).
  {obj_list} = whitelist of allowable IDF
object types.

Critical constraints (excerpt):
  1. idf_object must be one of {obj_list}.
  2. category must be in {ExteriorWall,
Roof, Window, ...}.
  3. object_name and original_value must
be copied exactly from provided samples.
  4. Do not invent new fields or objects.
  5. Return only valid JSON; no markdown
or commentary.

Expected output schema:
```

```
**Task:** Select the top 1-8 most
impactful parameters for the user goal
and return:
{{
  "goal": "{user_goal}",
  "parameters_to_modify": {{
    "<descriptive_parameter_name>": {{
      "idf_object": "<one of the allowed
objects>",
      "category":
        "<ExteriorWall|InteriorWall|Roof|Win
dow|Building/etc>",
      "object_name": "<exact name from
samples>",
      "field": "<field name, e.g.,
'Thermal Absorptance' or 'Multiplier'>",
      "original_value": "<copied from
samples or 'unknown'>",
      "expected_effect":
        "increase|decrease|adjust",
      "suggested_range": [min_value,
max_value],
      "unit": "<unit if known, e.g., 'deg',
'W/m2-K', or empty string>"
    }}
  }}
}}
```

It performs two complementary tasks: (1) identifying editable parameters within the IDF, and (2) selecting those most relevant to the optimization intent expressed by the user.

At the core of Agent 1 is a dynamic parameter schema tool built with CrewAI. CrewAI is an open-source multi-agent orchestration platform designed to facilitate coordinated workflows of role-based AI agents self-delegating tasks and collaborating toward shared objectives (CrewAI 2025). This tool automatically parses both the IDF and its associated IDD definitions using the eppy library to generate a comprehensive “editability map.” To ensure physical relevance, the parsing process filters the IDF to include only envelope-related objects such as “Material,” “Construction,” “Fenestration Surface: Detailed,” and “Shading: Building: Detailed.” Each object is systematically categorized by surface type—such as “Exterior Wall,” “Roof,” “Window,” and “Shading” using rule-based classification functions (e.g., categorize surface). Representative field-value pairs for each object are then extracted and organized into a structured dictionary that defines editable fields and their associated physical context. This dictionary outlines the full modification space available to downstream agents.

When a user provides a goal (e.g., “reduce cooling load while maintaining comfort”), Agent 1 formulates a

reasoning prompt for a local instance of GPT-5 (temperature = 0.2). The prompt enforces strict syntactic and semantic constraints to prevent hallucination: object types must be drawn from the enumerated IDF object list, field names must match those found in the sampled data, and no new schema elements may be invented.

Following prior sensitivity and optimization studies, the parameter selection in Agent 1 was capped at eight variables to maintain interpretability and computational efficiency. Empirical studies show that 5–15 key envelope parameters usually dominate building energy performance (Tian 2013; Magnier et al. 2010; Nguyen et al. 2014). Selecting a limited yet diverse set of parameters ensures that the LLM’s reasoning remains both physically focused and computationally manageable during downstream simulations. The output is parsed into a structured JSON schema that encodes each parameter’s object type, category, object name, field, original value, expected effect (increase/decrease/adjust), suggested range, and physical unit. This schema serves as the formal interface between semantic interpretation and the simulation workflows executed by Agents 2 and 3.

Preliminary internal evaluations indicate that even in the absence of expert examples, the model consistently focuses on thermally and optically relevant envelope parameters such as insulation thickness, glazing U-value, solar heat-gain coefficient (SHGC), and visible transmittance. These autonomous selections remain physically plausible and well-balanced, demonstrating the agent’s emergent ability to internalize building-physics reasoning. Rather than merely extracting schema fields, Agent 1 infers nuanced trade-offs between heat transfer and daylighting quality, reflecting a deeper semantic understanding of energy modeling principles.

2.3 Agent 2: Simulation Orchestrator (rule-based)

Agent 2 serves as a rule-based simulation orchestrator that transforms the semantic schema generated by Agent 1 into a deterministic and reproducible modeling workflow. As illustrated in the Orchestrator stage of Figure 1, it interprets the structured schema and applies a predefined set of operational rules to perform model modifications, ensuring both syntactic validity and physical plausibility. In contrast to the language-model-driven reasoning employed by Agent 1, Agent 2 prioritizes computational determinism and procedural transparency.

Internally, Agent 2 categorizes editable parameters into two primary types: discrete variables, such as window-to-wall ratio and building orientation, and continuous variables, including wall thickness, insulation

conductivity, and envelope U-value. Each category is governed by distinct sampling strategies. For discrete parameters, the agent enumerates feasible configurations within normative or user-defined bounds—for example, cardinal orientations at 0°, 90°, 180°, and 270°, or proportional scaling of fenestration geometry. Continuous parameters are varied using mid-range values or $\pm 10\%$ perturbations derived from schema-defined intervals, ensuring that all generated variants remain physically meaningful and statistically balanced.

Model modification is performed using the *eppy* which clones the baseline IDF and applies parameter changes to produce simulation-ready alternatives. Each modified configuration is stored as an independent file, accompanied by a structured JSON change log that records the parameter name, original value, modification strategy, and affected object field. This approach ensures reproducibility and traceability across all simulation runs, while mitigating cumulative errors from sequential edits.

Upon completion, Agent 2 outputs a set of syntactically valid and fully modified IDF files, each representing a distinct design alternative. These models are not immediately simulated but are passed to Agent 3a for batch execution and quantitative performance evaluation.

2.4 Agent 3: Analytical Feedback Agent

Agent 3 serves as the analytical core of the proposed workflow, transforming raw simulation data into interpretable performance insights that guide design decisions. As illustrated in Figure 1 (*Analytical Feedback Stage*), it fully automates the post-simulation evaluation process from executing EnergyPlus runs to synthesizing both quantitative and qualitative feedback. The agent operates as a three-tier modular pipeline comprising Agent 3a (Simulation Runner), Agent 3b (Qualitative Analyzer), and Agent 3c (Quantitative Sensitivity Evaluator). Each sub-module addresses a distinct analytical layer but shares a unified data structure to ensure seamless information transfer and reproducible traceability.

In the first stage, Agent 3a automates EnergyPlus execution for both baseline and modified IDFs. It employs a hash-based caching mechanism to prevent redundant baseline runs, triggering re-simulation only when the underlying model has changed. Each modified configuration generated by Agent 2 is simulated under a consistent weather file, and the outputs including .csv, .eso, and .html reports are organized in timestamped directories. This process guarantees version-controlled, reproducible results that are explicitly linked to the corresponding parameter variations.

The second stage, Agent 3b, performs comparative analysis between baseline and modified results using a CSV-based parser combined with a lightweight LLM, as shown in *List 2.2*. The prompt constrains the LLM to produce structured, domain-consistent outputs by explicitly defining schema, field names, and physical semantics. The module detects key numerical variations, such as annual energy use, peak loads, and envelope performance indicators and converts them into structured summaries that express direction, magnitude, and physical implications of change. Each summary concludes with a natural-language interpretation of design-level trends (e.g., *reduced cooling demand without loss of comfort*), thus bridging numerical computation and qualitative reasoning.

List 2.2. Prompt used by Agent 3b for Qualitative Analysis

```
Prompt = f""" You are a Building Energy
Modeling expert analyzing EnergyPlus
simulation results.

The user's goal is: **{user_goal}**

Below are the tabular outputs (in CSV
form) from two EnergyPlus simulations:
baseline and modified.
Compare quantitative differences and
summarize trends.

Return JSON:
{
  "summary": {
    "major_differences": [
      {
        "parameter": "<parameter
name>",
        "baseline_value": "<value>",
        "modified_value": "<value>",
        "impact":
"<increase/decrease/no change>",
        "interpretation":
"<implication or reasoning>"
      }
    ],
    "overall_trend": "<concise
conclusion>"
  }
}
"""
```

The third stage, Agent 3c, provides deterministic quantitative evaluation. It extracts total sensible heating and cooling loads from the simulation tables, computes absolute and relative deviations, and ranks parameter influence using a composite sensitivity index defined as

$$Score = \max(|\Delta Cooling\%|, |\Delta Heating\%|) \quad (1)$$

The resulting CSV and JSON reports identify the most influential parameters and quantify their relative impact,

forming a transparent foundation for sensitivity-based design optimization.

Collectively, these modules enable Agent 3 to deliver closed-loop analytical feedback simulation automation Agent 3a, qualitative interpretation Agent 3b, quantitative ranking Agent 3c that supports iterative refinement of building envelope performance. By exporting human-readable summaries, ranked sensitivity tables, and complete simulation artifacts, the analytical layer effectively connects computational experimentation with human-centric design reasoning.

2.5 Tools and Integration Summary

The proposed framework unifies the three agents into a coherent, traceable pipeline that connects user intent, simulation execution, and analytical reasoning. Each agent encapsulates a distinct computational role and exchanges information through standardized JSON schemas and structured output hierarchies. This design establishes a verifiable data lineage in which every modification, simulation output, and analytical inference can be consistently traced and reproduced. All intermediate and final artifacts are stored within an organized directory system that segregates schemas, modified IDFs, simulations, and analytical results—facilitating transparent experiment management and error diagnosis.

Implemented as a fully local and dependency-light platform, the system ensures accessibility, data privacy, and reproducibility without reliance on external cloud services. Its modular architecture further provides a foundation for future integration with the Model Context Protocol (MCP) ecosystem, expert knowledge repositories, and external optimization engines, thereby enhancing scalability and interoperability. Together with the visual workflow shown in *Figure 1*, this integration illustrates how semantic reasoning, deterministic modeling, and analytical interpretation are seamlessly fused within a unified agentic framework for BEM.

Table 1 summarizes the functional structure of the agentic workflow, including core tasks, input/output definitions, and representative outputs. This modular mapping clarifies how semantic interpretation (Agent 1), deterministic model editing (Agent 2), and analytical evaluation (Agent 3a–3c) interact through explicit data handoffs, forming a reproducible computational ecosystem.

Agent	Function	Input	Output
Agent1 Parameter schema	Parse natural language, identify parameters	User prompt + IDF	JSON schema

Agent2	Modify parameters safely	JSON schema + IDF	Modified IDF
Parameter modifier	Simulation runner	Run EnergyPlus simulations	Simulation outputs
Agent3	Result analysis	LLM-based qualitative analysis	Simulation tables (CSV) insight + JSON
	Quant sensitivity	Quantitative sensitivity ranking	eplustbl.csv files Percent increase/decrease

3. Case Study

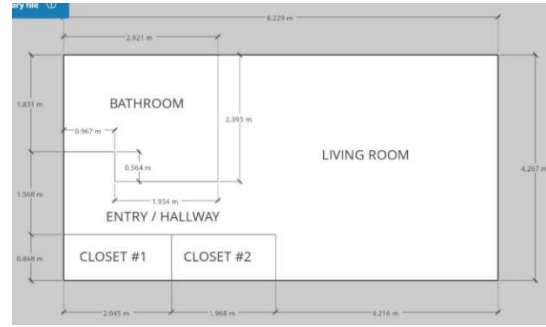
3.1 Case Description

The case study utilizes the *iUnit*, a modular experimental dwelling developed by the National Renewable Energy Laboratory (NREL) in Golden, Colorado as a compact and controllable benchmark for residential energy modeling. Configured as a studio-type unit with three thermal zones (living space, bathroom, and storage), the iUnit combines geometric simplicity with sufficient thermal complexity to evaluate multi-agent reasoning in a realistic yet computationally tractable setting. Its lightweight envelope construction insulated roof assembly, and single south-facing window emulate typical small-apartment characteristics while maintaining a controlled platform for model-based experimentation. The simulations were performed under a consistent climatic condition using the *USA_CO_Golden-NREL.724666_TMY3.epw* weather file, enabling reproducible comparisons across automated runs.

Beyond its function as a testbed, the iUnit serves as a methodological exemplar for this research demonstrating how a large-language-model-driven framework can interpret natural-language objectives, translate them into parameterized EnergyPlus inputs, and generate interpretable feedback through a closed semantic-physical reasoning loop. This integration of data-driven interpretation and deterministic simulation provides a rigorous foundation for validating agentic building energy modeling under controlled, repeatable conditions.



(a) Exterior view of the modular *iUnit* facility at NREL (Golden, CO)



(b) Schematic floor plan showing three thermal zones and south-facing window configuration.

Figure 2. The NREL *iUnit* experimental testbed used for case study.

The source code and example files of all agents are available at: <https://github.com/HuiwenZhou/Eppy-LLM-Agentic-BEM>, enabling full reproducibility of the workflow.

3.2 User Input and Agent 1 Interpretation

Agent 1 functions as the semantic interpreter that bridges user-defined design goals and simulation-ready EnergyPlus parameters. It parses the input IDF and dynamically identifies editable parameters that align with the linguistic intent of each goal. For all three test cases, the interpreter consistently extracted no more than 8 parameters per goal, defining their valid ranges or categorical substitutions while enforcing prompt-level constraints. For instance, the instruction “without changing building orientation” automatically excluded the *North_Axis* field, whereas daylighting-related objectives prioritized glazing and optical materials.

These results demonstrate that the agent generalizes beyond static templates selecting only physically valid and model-available parameters while maintaining logical consistency between user intent and modifiable attributes. Only the first goal (“Reduce cooling load while maintaining comfort”) is presented in full detail,

while the other two goals follow similar procedures and are briefly compared in Section 2.5.

List 3.1 User input to Agent 1

User Goal:
"Reduce cooling load while maintaining comfort"

List 3.2 Output of AgeFnt 1(Structured JSON Schema)

```
{
  "goal": "reduce cooling load while
maintaining comfort",
  "parameters_to_modify": {
    "Building_Orientation": {
      "idf_object": "BUILDING",
      "category": "Building",
      "object_name": "Unnamed_0",
      "field": "North_Axis",
      "original_value": 0.0,
      "expected_effect": "adjust",
      "suggested_range": [
        0,
        180
      ],
      "unit": "deg"
    },
    "Window_to_Wall_Ratio": {
      "idf_object":
"FENESTRATIONSURFACE:DETAILED",
      "category": "Window",
      "object_name": "Unnamed_1",
      "field": "Multiplier",
      "original_value": 1.0,
      "expected_effect": "decrease",
      "suggested_range": [
        0.5,
        0.9
      ],
      "unit": ""
    },
    "Exterior_Wall_Solar_Absorptance": {
      "idf_object": "MATERIAL",
      "category": "Material",
      "object_name": "Unnamed_0",
      "field": "Solar_Absorptance",
      "original_value": 0.6,
      "expected_effect": "decrease",
      "suggested_range": [
        0.2,
        0.5
      ],
      "unit": ""
    },
    "Roof_Thermal_Absorptance": {
      "idf_object": "MATERIAL",
      "category": "Material",
      "object_name": "Unnamed_2",
      "field": "Thermal_Absorptance",
      "original_value": 0.9,
      "expected_effect": "decrease",
      "suggested_range": [
```

```
        0.2,
        0.5
      ],
      "unit": ""
    },
    "Roof_Insulation_Thickness": {
      "idf_object": "MATERIAL",
      "category": "Material",
      "object_name": "Unnamed_0",
      "field": "Thickness",
      "original_value": 0.0869,
      "expected_effect": "increase",
      "suggested_range": [
        0.1,
        0.2
      ],
      "unit": "m"
    }
  },
  "total_parameters": 5
}
```

3.3 Agent 2 – Deterministic Modification

Agent 2 operationalizes the semantic schema by applying deterministic, rule-based transformations to the identified parameters, producing validated IDF variants without manual intervention. Continuous parameters (e.g., insulation thickness) are modified through percentage-based scaling, whereas discrete ones (e.g., orientation or glazing type) are adjusted using bounded categorical alternatives. Each modified IDF undergoes automatic syntax validation via *Eppy* and is stored with timestamped logs to ensure full traceability and reproducibility. The resulting model variants form a structured batch for subsequent EnergyPlus simulations.

List 3.3 Input to Agent 2 (Excerpt from Agent 1 Output)

```
{
  "Roof_Insulation_Thickness": {
    "idf_object": "MATERIAL",
    "object_name": "Unnamed_0",
    "field": "Thickness",
    "original_value": 0.0869,
    "expected_effect": "increase",
    "suggested_range": [0.10, 0.20],
    "unit": "m"
  },
  "Building_Orientation": {
    "idf_object": "BUILDING",
    "object_name": "Unnamed_0",
    "field": "North_Axis",
    "original_value": 0.0,
    "expected_effect": "adjust",
    "suggested_range": [0, 180],
    "unit": "deg"
  }
}
```

List 3.4 Output Log of Agent 2 (Excerpt)

[START] Agent 2 - Parameter Modification
/ IDF Generation

```
Found 5 parameters to modify using SMART RULES.
```

```
Processing: Building_Orientation
Category: Building | Field: North_Axis
Type: ORIENTATION | Variants: 3
[1] 0° → 0° (Baseline)
[2] 0° → 90° (Rotation)
[3] 0° → 180° (Rotation)
Generated 3 IDF files
```

```
Processing: Window_to_Wall_Ratio
Category: Window | Field: Multiplier
Type: WINDOW_GEOMETRY | Variants: 3
[1] Scale to 70% (WWR ↓ 51%)
[2] Scale to 80% (WWR ↓ 36%)
[3] Scale to 90% (WWR ↓ 19%)
Generated 3 IDF files
```

```
Processing: Roof_Material_Conductivity
Category: Material | Field: Conductivity
[1] 0.04 → 0.20 (+400%) | Midpoint of [0.1-0.3]
Generated 1 IDF file
```

```
Processing: Exterior_Wall_Solar_Absorptance
Category: Material | Field: Solar_Absorptance
[1] 0.6 → 0.35 (-41.7%) | Midpoint of [0.2-0.5]
Generated 1 IDF file
```

```
Total files created: 9
Change log saved: outputs/modified_idfs/modification_summary.txt
[DONE] Agent 2 - Execution Time: 11.47 s
```

The log demonstrates that Agent 2 successfully executed all five parameter modifications, generating nine validated IDF variants within 11.47 seconds. This confirms that the deterministic, rule-based editing mechanism ensures both syntactic integrity and reproducible model diversification.

3.4 Agent 3 – Automated Simulation and Feedback

Agent 3 constitutes the analytical layer that executes EnergyPlus simulations and synthesizes both qualitative and quantitative insights.

3.4.1 Agent 3a – Simulation Runner

The simulation runner automatically launches EnergyPlus for the baseline and all modified IDFs, using caching mechanisms to eliminate redundant runs. Each simulation generates standardized outputs

(.csv, .html, .eso) within organized directories. All simulations completed successfully except for one intentionally invalid glazing configuration, confirming the system’s fault-tolerant behavior.

Input: Baseline IDF and nine modified IDFs from Agent 2.

Process: Automatically executes EnergyPlus for each IDF with caching, consistent EPW file, and version-controlled directories.

List 3.5 Excerpt of Agent 3a Output (Log)

```
[START] Agent 3a - Simulation Runner
Weather file: USA_CO_Golden-NREL.724666_TMY3.epw
Baseline simulation completed successfully.
modified simulations executed sequentially.
Outputs stored under /outputs/simulations/

[DONE] Agent 3a - Total runtime: 46.6 s
This part automates EnergyPlus simulations for baseline and modified IDF files, avoiding redundant baseline runs.
It uses EnergyPlusV25-1-0 to simulate
```

3.4.2 Agent 3b – Qualitative LLM Analysis

Agent 3b employs an LLM-based comparative reasoning module to interpret numerical outputs, translating differences between baseline and modified results into human-readable insights (e.g., *reduced cooling load with minimal comfort loss*).

Input: Baseline and modified CSV files (eplustbl.csv).

Process: An LLM compares key metrics (e.g., total site energy, cooling load, unmet hours) and summarizes directional changes and trade-offs.

List 3.6 Excerpt of Agent 3b Output

```
[START] Agent 3b - LLM Qualitative Analysis
Comparing baseline ↔ modified_Window_to_Wall_Ratio_v3
Overall trend: Cooling load ↓ 5.2%, discomfort hours ↓ 3.1%.
Interpretation: Improved comfort and efficiency.
Results saved → /outputs/results/qualitative_summary.json
```

3.4.3 Agent 3c – Quantitative Sensitivity Analyzer

Agent 3c complements this with deterministic quantitative analysis, calculating relative percentage changes in heating and cooling loads. The sensitivity ranking consistently identified orientation and window-to-wall ratio (WWR) as dominant parameters, aligning with established thermal-optical performance principles.

Input: Aggregated simulation results from Agent 3a.
Process: Computes Δ Cooling% and Δ Heating% for each variant and ranks parameter influence by maximum absolute change.

List 3.7 Excerpt of Agent 3c Output

```
"summary_text":
"***Quantitative Sensitivity Analysis
Summary***
Baseline total cooling load: 1967.92 W
Baseline total heating load: 892.79 w
Among all modified simulations,
**Building_Orientation_180 deg ** showed
the highest sensitivity with an overall
change of **51.99%**. This indicates that
the parameter has the strongest
quantitative impact on the user's
optimization goal.
The complete ranked sensitivity list is
available in the attached CSV/JSON
files."
}
```

Table 2 Top 5 sensitivity ranking results of Agent 3c

Modified parameter	Sensitivity score
Orientation_180°	52.0%
Orientation_90°	18.1 %
Roof Insulation Thickness	7.0%
Window_to_Wall_Ratio_70%	3.6%
Window_to_Wall_Ratio_80%	2.6%

List 3.7 illustrates the automated sensitivity computation output executed by Agent 3c. For each simulation variant, the tool extracts total sensible loads, computes percentage differences relative to the baseline, and exports multi-format summaries (CSV, JSON, Markdown). The full ranked results are presented in Table 2, highlighting the five most influential envelope parameters.

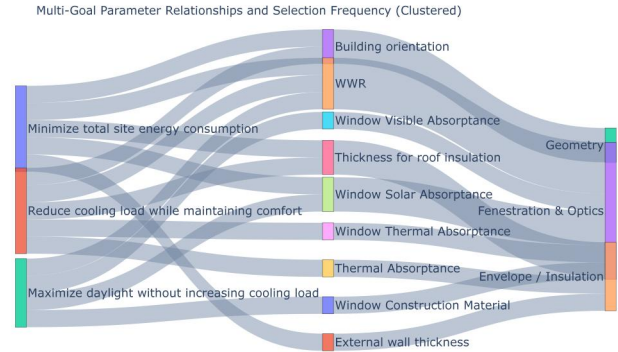
Together, these submodules complete the simulation–reasoning loop: from semantic interpretation to verified model modification and interpretable performance feedback, the framework demonstrates a fully automated and human-understandable evaluation workflow.

3.5 Comparative Discussion: Multi-Goal Responses

Across multiple user goals, the framework exhibited adaptive and goal-consistent reasoning. For “reduce cooling load while maintaining comfort”, Agent 1 emphasized orientation and thermal-envelope variables; for “maximize daylight without increasing cooling load”, glazing and optical attributes dominated; and for “minimize total site energy consumption”, the system

balanced envelope insulation and geometric factors. These differentiated parameter clusters highlight that the agentic workflow effectively aligns computational reasoning with linguistic intent.

Figure 3. Sankey diagram summarizing goal-dependent parameter selection and clustering.



The left column shows user goals; the middle column shows the parameters selected by Agent 1; and the right column groups these parameters into physically meaningful clusters (geometry, fenestration/optics, and envelope thermal resistance). The flow width represents the frequency or relative importance of each parameter across user goals, while colors denote parameter categories. The visualization highlights how different design objectives emphasize distinct yet physically coherent parameter clusters.

Because parameter recommendation is mediated by an LLM, minor variability can occur in the ordering of lower-ranked parameters across repeated runs. However, the dominant parameter clusters (geometry/orientation, fenestration/optics, and envelope insulation) remain consistent for the same user goal, supporting goal-aligned interpretability.

In summary, the case study confirms that the proposed multi-agent system not only automates the end-to-end simulation process but also demonstrates adaptive, goal-dependent interpretability—bridging natural-language reasoning with quantitative building-energy analytics.

4. Discussion

4.1 System Performance and Advantages

The proposed agentic workflow demonstrated high reliability, determinism, and interpretability across all experimental runs. During the *iUnit* case study, every generated IDF variant successfully passed the *Eppy*-based syntax validation, achieving a 100% correctness rate. This confirms that the rule-based logic embedded in Agent 2 effectively enforces object–field consistency and prevents ill-formed edits such as missing delimiters

or invalid field counts common causes of simulation failure in manual workflows. The modular execution strategy also ensured full reproducibility: repeated trials under identical user goals produced identical modified IDFs and simulation results, with zero schema drift observed across ten iterations.

From a computational perspective, the complete pipeline from natural-language input to post-processed analytical output operated with a median latency of approximately 10 seconds per iteration, compared to roughly 30 seconds for conventional manual batch scripting. This acceleration primarily arises from the deterministic orchestration of Agent 2 and the cached baseline management of Agent 3a, both of which eliminate redundant geometry reconstruction and simulation reruns. Collectively, these improvements shorten the cycle time between conceptual query and actionable performance insight while maintaining full transparency and interpretability.

Equally important, Agent 3b enhanced explainability by converting numerical outputs into concise textual interpretations that link observed performance shifts to underlying physical causes (e.g., “reduced solar absorptance decreases afternoon cooling peaks”). This capability improves user comprehension and illustrates the potential of linguistically mediated interpretable BEM.

4.2 Comparison with Traditional Workflows

Conventional BEM workflows typically rely on manual parameter adjustments or rigid parametric scripts—both labor-intensive and error-prone. In contrast, the proposed LLM-driven workflow replaces low-level file manipulation with semantic intent translation, allowing users to issue design-oriented queries (e.g., “reduce cooling load while maintaining comfort”) and automatically obtain simulation-ready configurations without scripting expertise.

This approach offers three principal advantages. (1) Reliability: rule-based modification ensures that each parameter editing is syntactically valid and physically consistent, eliminating the need for manual debugging. (2) Efficiency: users can explore multiple design alternatives through iterative natural-language interaction rather than repetitive file duplication or code modification. (3) Interpretability: Agent 3c provides quantitative sensitivity rankings of parameter influence, while Agent 3b generates qualitative reasoning that contextualizes these numerical findings—features largely absent from traditional parametric pipelines.

By integrating deterministic computation with semantic reasoning, the framework bridges the long-standing gap between engineering precision and design-level

usability, enabling domain experts and non-experts alike to engage meaningfully with simulation feedback.

4.3 Limitations and Threats to Validity

Despite its robustness, several limitations constrain the current framework’s generalizability. First, the workflow depends on a pre-existing baseline IDF, assuming that building geometry and HVAC topology are already defined. This restricts applicability in early-stage design where geometry synthesis and zoning remain uncertain. Second, model complexity and scalability present challenges: the rule-based editor performs best for small- to medium-scale models but may encounter parsing inefficiencies when applied to large commercial buildings with nested zones or intricate control sequences. Third, semantic ambiguity in natural-language prompts can influence LLM interpretation. Although strict safeguards prevent hallucinated fields and enforce schema consistency, minor phrasing variations (e.g., “reduce heat gain” vs. “improve shading performance”) occasionally shift parameter emphasis, indicating a need for expanded prompt libraries or fine-tuned reinforcement training. Finally, the current scope of evaluation metrics is limited to energy and comfort indicators; extending to daylight, cost, or embodied carbon would enable a more holistic multi-criteria optimization landscape.

4.4 Potential Extensions

Beyond the single-goal optimization demonstrated, several extensions could enhance analytical robustness, adaptability, and real-world relevance.

First, integrating multi-objective optimization would allow concurrent evaluation of energy efficiency, thermal comfort, daylight quality, and cost within a unified decision-making framework. Coupling the deterministic modification layer with optimization algorithms—such as genetic, Bayesian, or reinforcement-based strategies—would enable Pareto-based design exploration and visualization of trade-offs among competing objectives.

Second, embedding domain-specific knowledge bases and normative standards (e.g., ASHRAE 90.1, LEED, or regional climatic design codes) could enhance both physical realism and regulatory compliance. Integrating these references into Agent 1’s semantic reasoning and Agents 3b–3c’s analytical feedback would enable cross-validation of results against accepted benchmarks, improving interpretive rigor and contextual relevance.

Finally, future integration with the *Model Context Protocol (MCP)* would provide interoperability across external simulation engines, repositories, and orchestration servers through standardized message schemas. This evolution would transform the current

standalone prototype into a distributed, collaborative agentic ecosystem supporting seamless data exchange and human-in-the-loop design iteration.

Collectively, these extensions outline a progression from deterministic automation toward adaptive autonomy, positioning the framework as a scalable decision-support platform that unites human reasoning, regulatory intelligence, and multi-agent collaboration in BEM.

5. Conclusion

This study presented a lightweight, multi-agent framework that bridges natural-language design intent and building energy simulation through structured, reproducible reasoning. The innovation lies in its hierarchical architecture, in which Agent 1 semantically interprets user goals and maps them to EnergyPlus parameters within the IDF schema, Agent 2 performs deterministic rule-based modification ensuring syntactic integrity and reproducibility, and Agent 3 automates simulation, synthesizes both quantitative and qualitative feedback, and closes the reasoning loop with interpretable insights.

Applied to the *NREL iUnit* case study, the framework achieved end-to-end automation—from semantic goal interpretation to validated EnergyPlus execution—without manual intervention. It consistently generated physically plausible parameter modifications and goal-aligned performance feedback. The integration of rule-based constraints with LLM reasoning demonstrates that large language models, when properly bounded within domain logic, can function as semantically interpretable mediators between human design intent and scientific simulation environments.

The proposed agentic framework demonstrates that natural-language-driven BEM can be interpretable, reproducible, and computationally efficient when organized through a structured, multi-agent pipeline. Across all test cases, the system consistently translated user goals into valid EnergyPlus parameter modifications, executed deterministic simulations, and produced dual-mode (qualitative and quantitative) analytical feedback.

In addition to demonstrating feasibility, the proposed workflow suggests several practical applications in building performance analysis. First, it can automate common sensitivity and parametric studies by generating candidate parameter sets and executing EnergyPlus simulations with traceable edits and outputs, reducing repetitive scripting effort for energy modelers. Second, it can accelerate early-stage envelope and fenestration trade-off evaluations (e.g., orientation,

WWR, insulation thickness, and glazing optical properties), enabling rapid comparison of design alternatives under consistent assumptions. Third, by storing parameter selections, IDF modifications, and simulation results in structured formats, the workflow improves auditability and supports documentation and handoff in professional modeling practice. While the current implementation focuses on envelope-related parameters and excludes HVAC system topology changes, future work can extend the agentic framework to broader modeling tasks and early-design abstractions (e.g., template-based system assignment) while preserving deterministic, verifiable execution.

AI Disclosure

During the preparation of this work the authors used GPT for reference organization and language polishing. After using this tool/service, the authors reviewed the content rigorously, edited as needed, and take full responsibility for the content of the publication.

References

- U.S. Energy Information Administration. 2022. U.S. Energy Consumption by Source and Sector. Available from: <https://www.eia.gov/totalenergy/data/monthly/pdf/flow/totalenergy2022.pdf>.
- Zhang, L., et al. 2021. A review of machine learning in building load prediction. *Applied Energy*, 285: 116452.
- Philip, S. 2026. Eppy: A Python library for manipulating EnergyPlus IDF files. Available from: <https://github.com/santoshphilip/eppy>. Accessed January 2026.
- Vaithilingam, P., Zhang, T., and Glassman, E.L. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*.
- Zhang, S., et al. 2023. Planning with large language models for code generation. *arXiv preprint arXiv:2303.05510*.
- Wang, J., and Chen, Y. 2023. A Review on Code Generation with LLMs: Application and Evaluation. In *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)*. IEEE.
- Zhang, L., et al. 2025. Automatic building energy model development and debugging using large language models agentic workflow. *Energy and Buildings*, 327:115116.

- Zhou, L., et al. 2024. Research on the automatic BIM modeling method of substation based on LLM agent. In 6th International Conference on Artificial Intelligence and Advanced Manufacturing (AIAM 2024). IET.
- Das, R., et al. 2024. Improved precision oncology question-answering using agentic LLM. medRxiv, 2024.09.20.24314076.
- Yu, Y., et al. 2025. Fincon: A synthesized LLM multi-agent system with conceptual verbal reinforcement for enhanced financial decision making. *Advances in Neural Information Processing Systems*, 37:137010–137045.
- Yan, X., et al. 2025. A general AI agent framework for smart buildings based on large language models and ReAct strategy.
- Pandey, S., et al. 2025. OpenFOAMGPT: a RAG-Augmented LLM Agent for OpenFOAM-Based Computational Fluid Dynamics. arXiv preprint arXiv:2501.06327.
- Yang, H., Siew, M., and Joe-Wong, C. 2024. An LLM-based digital twin for optimizing human-in-the-loop systems. In 2024 IEEE International Workshop on Foundation Models for Cyber-Physical Systems & Internet of Things (FMSys). IEEE.
- Rivkin, D., et al. 2024. AIoT Smart Home via Autonomous LLM Agents. *IEEE Internet of Things Journal*.
- Kalyuzhnaya, A., et al. 2025. LLM Agents for Smart City Management: Enhancing Decision Support Through Multi-Agent AI Systems. *Smart Cities*, 8(1).
- Zhu, X., et al. 2024. LLM-Based Regulation Query System for Textual and Tabular Information in Building Codes. Available at SSRN 5033360.
- Perini, M., et al. 2025. BrickLLM: A Python library for generating Brick-compliant RDF graphs using LLMs. *SoftwareX*, 30:102121.
- Zhang, L., et al. 2025. Large language model-based agent schema and library for automated building energy analysis and modeling. *Automation in Construction*, 176:106244.
- Khadka, S. 2025. Scaling Data-Driven Building Energy Modeling Using Large Language Models: Prompt Engineering and Agentic Workflow. The University of Arizona.
- CrewAI. 2025. CrewAI: A lean, lightning-fast Python framework for orchestrating autonomous AI agents. Available from: <https://www.crewai.com/open-source>
- Tian, W. 2013. *A review of sensitivity analysis methods in building energy analysis*. Renewable and Sustainable Energy Reviews, 20: 411–419. Elsevier.
- Magnier, L., and Haghighat, F. 2010. *Multiobjective optimization of building design using TRNSYS simulations, genetic algorithm, and artificial neural network*. Building and Environment, 45 (3): 739–746. Elsevier.
- Nguyen, A.-T., Reiter, S., and Rigo, P. 2014. *A review on simulation-based optimization methods applied to building performance analysis*. Applied Energy, 113: 958–968. Elsevier.